Introduction to Back-end Development

Presented by:

Cristian Camilo Cortes Ortiz

Codecamp academy

Table of contents

Audience 3
Purpose 3
Introduction 3
Back-end Development Roles 4
DB Admin 4
Server Admin 4
Front-end, Back-end, and Full Stack 5
Front-end 5
Browser technologies 5
Mobile technologies 6
Back-end 7
Full stack 8
Server-side rendering 8
Web and API Fundamentals 9
HTTP 9
HTTP Methods 9
HTTP response codes 10
APIs 11
REST API 12
Domain 12
DNS 12
Endpoints 13
CRUD 14
API Architecture 15
Routers 15
Middleware 16
Controllers 17
Services 17
Models 18
Schemas 18
Databases 19
ORMs 20
Migrations 20
Seeders 20
API Security 21
Authentication and Authorization 21
Cookies and Sessions 21

JSON Web Tokens (JWT) 22
Configuration 23
Environment Variables 23
Infrastructure 24
Server 24
DevOps 25
Cloud 26
Scalability and Performance 27
Vertical scaling 27
Horizontal scaling 27
Replication 29
Cache 30
Task Queues 30
Real-time Communication 31
WebSockets 31
Socket.IO 32
Server implementation 32
Client implementation 33

Audience

This manual contains technical terminology and is intended for IT students or anyone interested in learning about the fundamentals of software engineering in web back-end development. It delves into concepts that will help you understand communication protocols, back-end application architecture, and other basic concepts.

Purpose

The purpose of this manual is to provide readers with a clear understanding of the essential concepts in back-end web development. It will cover key topics such as communication protocols, APIs, databases, scalability, and security, among others. This content.

Introduction

Back-end web development is a fundamental branch of software engineering responsible for managing business logic, communicating with databases, and implementing services that enable web applications to function. Throughout the evolution of technology, various tools, architectures, and methodologies have emerged that facilitate the creation of scalable, secure, and efficient systems. This document explores the fundamentals of back-end development, providing a solid foundation for understanding how server-side applications work and how they interact with other technologies.

Back-end Development Roles

The primary role of a Backend Developer is to write code related to:

- Business Rules.
- Validation.
- User Authorization.
- Database Connections.
- Code that will run on the server side.

A backend developer may also be associated with other types of roles, such as:

DB Admin

This person is responsible for managing a database, its policies, and how that database will be made available through the code and the security measures it should have.

Server Admin

This person is responsible for managing security on the servers, which is where the code runs.

Front-end, Back-end, and Full Stack

Front-end

It focuses on the rendering aspect of the graphical user interface, using clients such as:

- Web browsers.
- Mobile apps.
- IoT.

Browser technologies

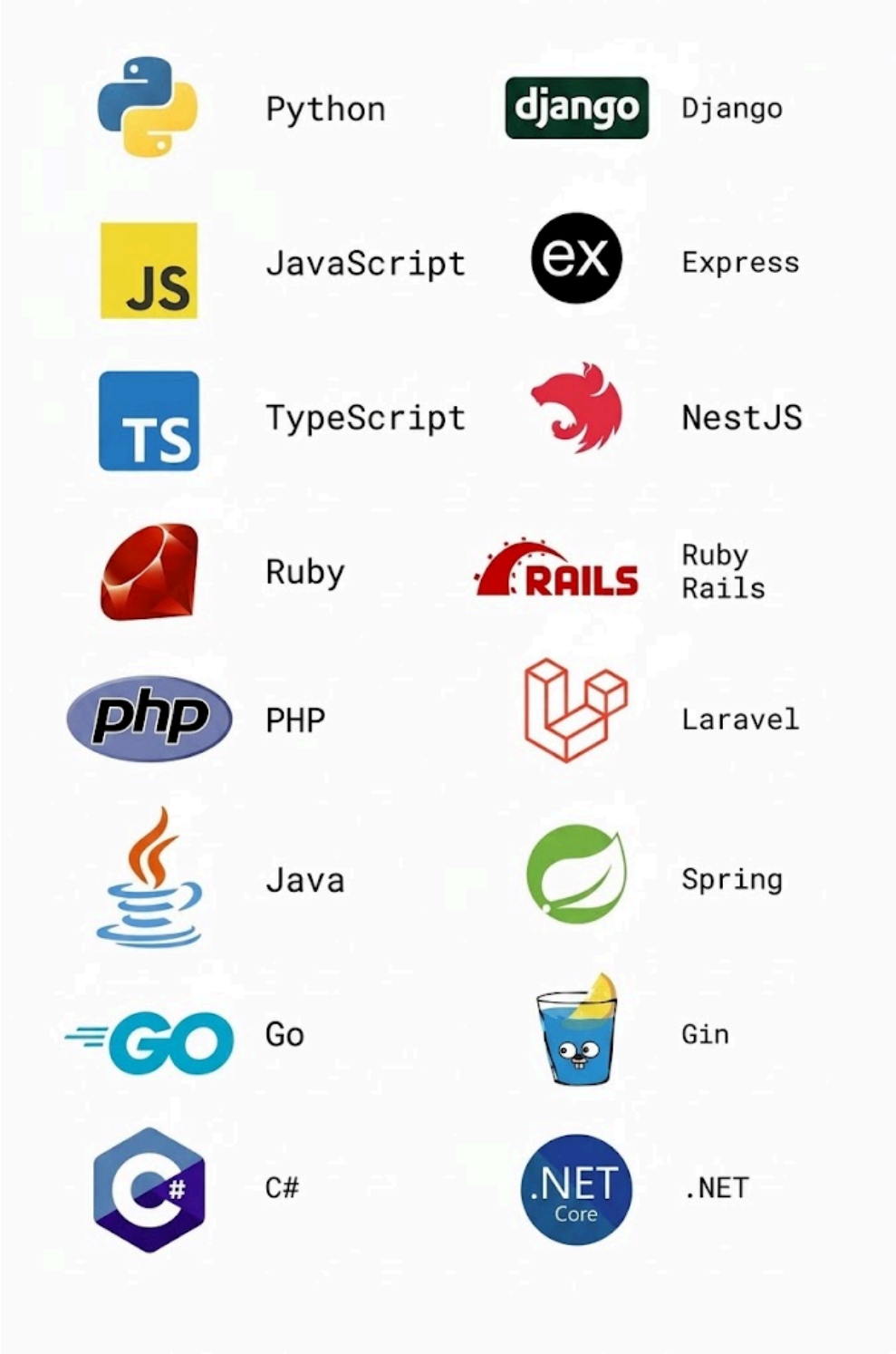


Mobile technologies



Back-end

It focuses on the overall layout and distribution of the services offered by the app.



Full stack

It focuses on both the front end and the back end; in other words, this role combines responsibilities related to both the client and the server.

Server-side rendering

Server-side rendering (SSR) is a technique used in web development where HTML pages are generated and rendered on the server before being sent to the client. Unlike client-side rendering, where the user's browser builds the page using JavaScript, SSR delivers fully rendered content from the server, thereby improving initial load time and search engine indexability. This is particularly beneficial for applications that require fast content delivery and better SEO performance. Frameworks such as:

- Next.js for React.
- Nuxt.js for Vue.js.

They facilitate the implementation of SSR, allowing developers to combine the best of both worlds: fast loading times and interactivity. Although SSR can increase server load and development complexity, it offers a faster and smoother user experience, especially on slow connections or devices with limited resources.

Server Response Formats:

- JSON.
- XML.
- Files.
- HTML.

Web and API Fundamentals

HTTP

HTTP (Hypertext Transfer Protocol) is an application-layer protocol used for communication between clients and servers over a network. It defines how a client, such as a web browser or application, sends requests to a server and how the server returns responses. In backend development, HTTP provides the fundamental communication mechanism used by APIs, allowing clients to request resources, submit data, update information, or perform other operations on a server. Each HTTP communication typically contains a request and a response, which include information such as the method, URL, headers, status code, and, when necessary, a request or response body.

HTTP Methods

HTTP methods define the type of operation that a client wants to perform on a resource exposed by a backend application. The most commonly used methods in API development are GET for retrieving data, POST for creating resources, PUT for replacing an existing resource, PATCH for partially updating a resource, and DELETE for removing a resource. For example, a GET request to /users can retrieve a collection of users, while a POST request to the same endpoint can create a new user. Using standardized HTTP methods allows APIs to communicate their intended operations clearly and consistently.

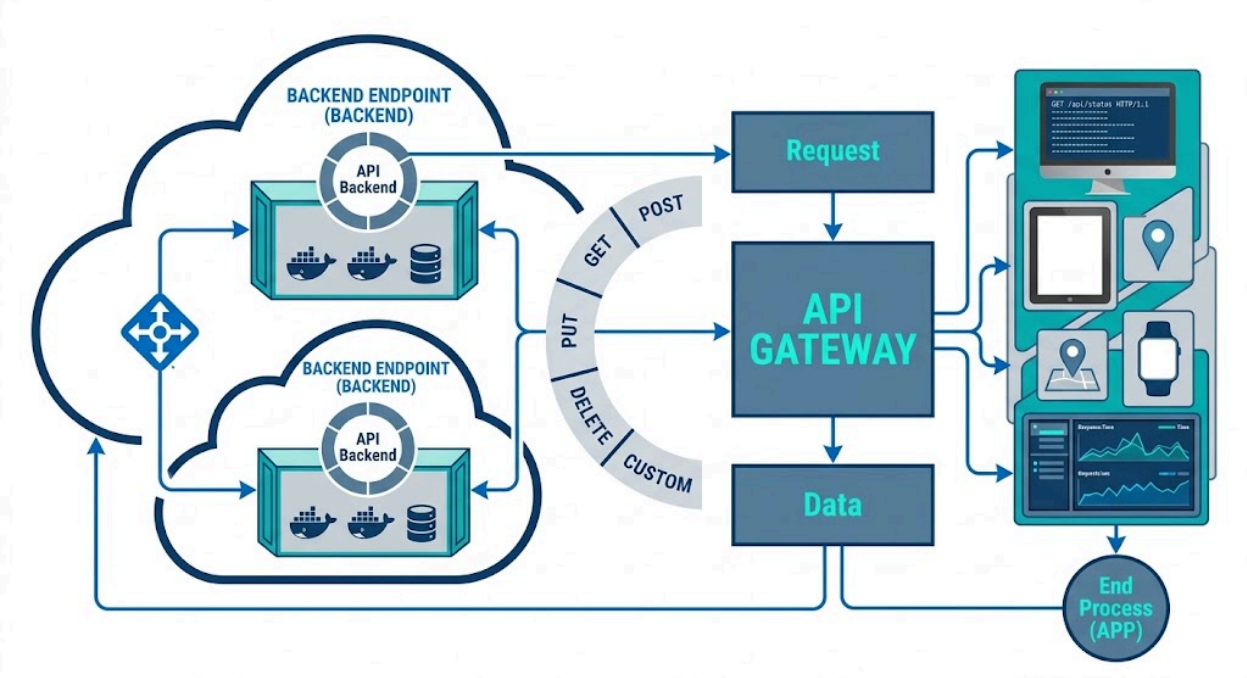
HTTP response codes

HTTP response codes are a set of status codes returned by a server in response to an HTTP request from a client. These codes indicate the outcome of the request and help developers and users understand what has happened. They are classified into five categories.

From	To	Type
100	199	Informative
200	299	OK
300	399	Redirect
400	499	Client Error
500	599	Server Error

APIs

APIs (Application Programming Interfaces) are sets of definitions and protocols that enable communication between different software applications. They act as intermediaries that facilitate integration and interaction between different systems, allowing independently developed applications to work together efficiently. APIs specify how requests and responses should be handled between software components, providing predefined methods and data that developers can use to implement functionality without needing to fully understand the underlying code. In web development, APIs are essential for creating dynamic and connected applications, facilitating the exchange of data and services between servers, databases, and clients.



REST API

REST APIs (Representational State Transfer Application Programming Interfaces) are a type of API that follows the principles of REST architecture, a standard for designing scalable and efficient web services. REST uses basic HTTP operations—such as GET, POST, PUT, and DELETE—to interact with resources represented in JSON or XML format. Each resource is identified by a unique URI (Uniform Resource Identifier). REST APIs are known for their simplicity and flexibility, which allow developers to build applications that can easily communicate with various services and platforms. Thanks to their design based on open standards, REST APIs facilitate interoperability between systems and are widely adopted in the development of modern web applications.

Domain

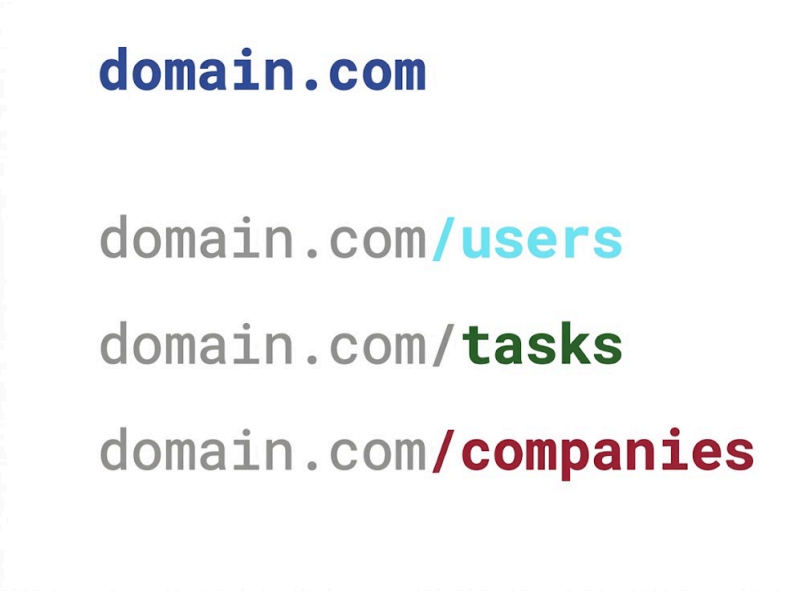
A domain is a human-readable name used to identify a server, application, or service on the internet. Instead of requiring users to remember an IP address such as 192.168.1.100, they can access an application using a domain such as domain.com. In backend development, the domain represents the main address through which clients can reach the application. For example, an API could be available at api.domain.com, where api is a subdomain used to identify the backend API.

DNS

The Domain Name System (DNS) is the system responsible for translating domain names into IP addresses. When a client requests domain.com, the DNS infrastructure determines which IP address is associated with that domain and allows the client to establish a connection with the corresponding server. DNS therefore acts as a distributed naming system that makes internet services accessible through meaningful names instead of numerical IP addresses.

Endpoints

An endpoint is a specific URL path exposed by a backend application through which a client can interact with a particular resource or operation. For example, an API could expose endpoints such as domain.com/users, domain.com/tasks, and domain.com/companies. Endpoints can also contain identifiers or nested resources, such as domain.com/users/15/tasks, where 15 identifies a specific user. Each endpoint is typically associated with one or more HTTP methods, such as GET, POST, PUT, PATCH, or DELETE, which determine the type of operation that the client can perform.



CRUD

CRUD (Create, Read, Update, Delete) is an acronym that describes the four basic operations required to interact with a database or data storage system. These operations are essential for handling and managing information in software applications. CRUD is an essential methodology in the development of web applications and database management systems, providing a clear and consistent structure for developing data-handling features. Implementing these operations efficiently and securely is crucial to ensuring the integrity and accessibility of information.

Operation	Method
CREATE	POST
READ	GET
UPDATE	PUT / PATCH
DELETE	DELETE

Endpoint	Method	Body	Response	Status
/products	GET	N/A	[{},{}...]	200
/products/:id	GET	N/A	{...}	200
/products	POST	{...}	{...}	201
/products/:id	PUT / PATCH	{...}	{...}	201
/products/:id	DELETE	N/A	✅	201

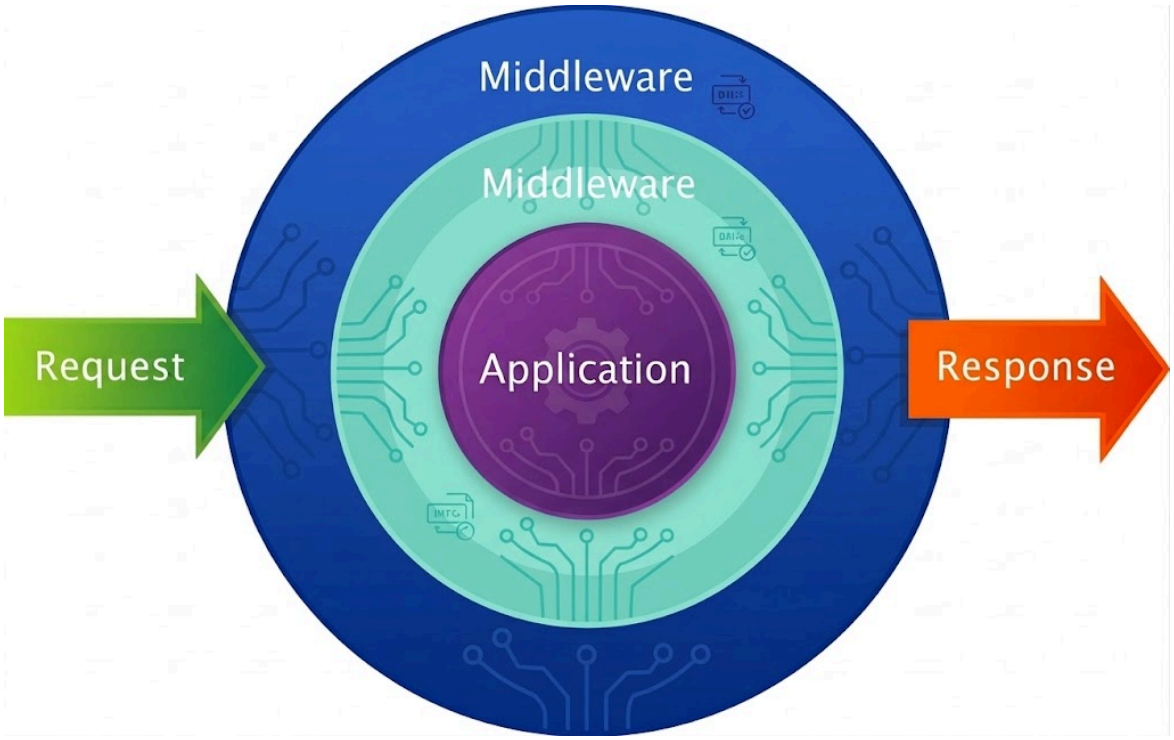
API Architecture

Routers

A route defines how a backend application responds to a specific HTTP request, while a router is the component responsible for organizing and registering multiple routes within the application. A route typically combines an HTTP method, a URL path, and a handler that determines what the server should do when that endpoint is requested. For example, a backend could define a GET /users route to retrieve users, and a POST /users route to create a new user. Routers help organize these routes by grouping related resources, making the application easier to structure, maintain, and scale. In larger applications, routes are often separated into modules such as users, authentication, products, or companies, with each router responsible for the endpoints related to a specific resource.

Middleware

Middleware is a component of a backend application that executes between the incoming client request and the final request handler or controller. It can inspect, modify, or process a request before it reaches its destination, and it can also process the response before sending it back to the client. Middleware is commonly used for tasks such as authentication, authorization, request logging, validation, error handling, CORS configuration, and request data processing. For example, an authentication middleware can verify whether a request contains a valid access token before allowing the request to access a protected endpoint.



Controllers

A controller is a component of a backend API responsible for handling incoming HTTP requests and coordinating the appropriate response. Controllers receive requests from clients through defined routes, extract relevant information such as parameters, query strings, request bodies, or authenticated user data, and then execute the necessary application logic or delegate that work to other components. For example, a UserController may contain operations for creating, retrieving, updating, and deleting users. In a well-structured API, controllers should primarily focus on managing the HTTP layer, while business logic is delegated to services or other specialized components. This separation of responsibilities makes the application easier to maintain, test, and scale as the number of resources and endpoints increases.

Services

Services are components of a backend API responsible for containing and executing the application's business logic. They are typically used to separate business rules and operations from controllers, allowing controllers to focus on handling HTTP requests and responses. For example, a UserService could contain operations for creating users, checking whether an email already exists, assigning roles, or retrieving user information from a repository or model. Services can also coordinate operations involving multiple models or external services. This separation of responsibilities makes the API easier to maintain, test, and reuse, especially as the application's business logic becomes more complex.

Models

Models are components of a backend API that represent the data and structure of the resources managed by the application. A model commonly defines the attributes of a resource, its relationship with other resources, and how its data is stored or retrieved from the database. For example, a User model may represent information such as a user's name, email, password, and creation date, while also defining relationships with other models such as Task or Company. Models provide a structured way for the application to work with persistent data and are commonly used by controllers or services when performing operations such as creating, retrieving, updating, and deleting records.

Schemas

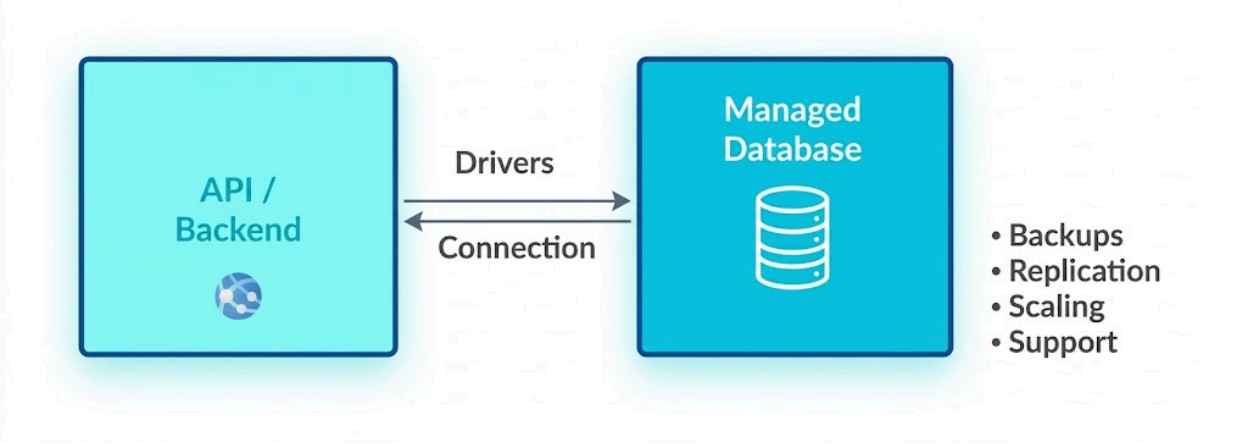
Schemas define the structure, data types, and rules that the data used by an API must follow. They can be used to describe the expected format of request bodies, query parameters, responses, or database documents, depending on the technology and architecture being used. For example, a user schema may specify that name must be a string, email must follow a valid email format, and age must be a number greater than or equal to 18. Schemas help APIs validate incoming data, ensure that responses follow a consistent structure, and make the contract between the client and the backend more predictable. They are particularly useful for preventing invalid or unexpected data from entering the application.

Databases

These are systems designed for the collection, storage, and management of data in a structured manner, enabling efficient access, retrieval, and manipulation of information. There are different types of databases, such as relational databases, which use tables to represent data and its relationships using SQL (Structured Query Language), and NoSQL databases, which are designed to handle large volumes of unstructured or semi-structured data using flexible models such as documents, graphs, key-value pairs, and columns. Databases are essential for:

- Enterprise Applications.
- Web Applications.
- Mobile Applications.
- Content Management Systems.
- Data Analytics.

And many other applications that require persistent storage and fast access to information. Choosing the right type of database and designing it appropriately are crucial for the performance, scalability, and reliability of the applications that depend on them.



ORMs

Object-relational mapping (ORM) is a programming technique that allows developers to interact with relational databases using object-oriented programming languages. ORMs act as intermediaries between the application and the database, translating operations on objects into SQL queries and vice versa. This abstraction simplifies data handling, as developers can work with objects and their methods instead of writing manual SQL queries. Among the most popular ORM tools is the use of ORM. Using an ORM improves development productivity, facilitates code maintenance, and promotes component reuse, although it can introduce some overhead and limitations compared to manually optimized SQL queries.

Migrations

Migrations are files used to define and manage changes to the structure of a database throughout the development of a backend application. They allow developers to create tables, define columns, establish relationships, modify existing structures, or remove database objects in a controlled and reproducible way. Instead of manually modifying the database, developers can create migration files that describe each change and execute them in a specific order. For example, a migration can create a users table with fields such as id, name, email, and password. Migrations help keep the database structure consistent across development, testing, and production environments and make it possible to track how the database schema evolves as the API grows.

Seeders

Seeders are components used in backend development to populate a database with predefined or generated data. They are especially useful during development and testing because they allow developers to quickly create the records required for an application to operate or demonstrate its functionality. For example, a seeder can create users, companies, products, or other related records with predefined values. Seeders can also be combined with data generators to create large amounts of realistic test data. By automating database population, seeders make it easier to reproduce consistent development environments and test API endpoints without manually inserting records into the database.

API Security

Authentication and Authorization

Authentication and authorization are two fundamental security mechanisms used in APIs to control access to protected resources. Authentication is the process of verifying the identity of a client or user, for example, by validating credentials, an API key, or an access token. Authorization takes place after authentication and determines what resources or operations the authenticated user is allowed to access. For example, an API may authenticate a user using a JWT and then authorize that user to access their own profile while restricting administrative endpoints to users with an administrator role. Together, authentication and authorization help APIs ensure that only verified users can access resources and that they can perform only the operations permitted by their assigned permissions.

Cookies and Sessions

These are mechanisms used in web development to manage and maintain the state of interactions between users and websites. Cookies are small text files stored in the user's browser that contain specific data about their visit, such as site preferences, authentication information, or browsing history. They allow websites to recognize the user on subsequent visits. Sessions, on the other hand, store information on the server and are linked to a unique identifier that is sent to the client in the form of a cookie or in the URL. Sessions are temporary and typically expire after a period of inactivity, ensuring greater security for sensitive data. Both mechanisms are essential for creating personalized user experiences and maintaining continuity in interactions within web applications.

JSON Web Tokens (JWT)

JSON Web Tokens (JWT) are a standardized mechanism commonly used in API development to securely transmit claims between a client and a server. A JWT is typically issued by the backend after a user successfully authenticates and contains information about that authentication session, such as the user's identifier, roles, or expiration time. The client can then include the token in subsequent requests, usually through the Authorization header using the Bearer scheme. The API can validate the token's signature and claims before allowing access to protected endpoints. JWTs are particularly useful for stateless authentication because the server can validate a token without necessarily maintaining a session record for every authenticated client:

- The header.
- The payload.
- The signature.

The header specifies the encryption algorithm used, the payload contains the claims, and the signature ensures the token's integrity using a cryptographic algorithm. JWTs are widely used for user authentication and authorization in web and mobile applications, enabling user verification and the exchange of information between the client and the server without the need to maintain state on the server. Thanks to their compact format and the ability to be verified using digital signatures, JWTs are ideal for microservice environments and distributed applications, facilitating secure and scalable access.

Configuration

Environment Variables

Environment variables are configuration values stored outside the application's source code and used to provide information that can change depending on the environment in which the API is running. They are commonly used to store values such as database connection information, API keys, authentication secrets, application ports, and external service URLs. For example, an API might use variables such as DATABASE_URL, JWT_SECRET, and PORT to configure its behavior without hardcoding these values directly into the source code. This approach makes applications easier to configure across development, testing, and production environments while also helping prevent sensitive information from being exposed in the source code or committed to a version control repository.

Infrastructure

Server

A computer system that provides resources, data, services, or programs to other computers, known as clients, over a network. Servers can perform various functions, such as:

- Website Hosting.
- Database Administration.
- Email Delivery.
- Application Execution.

There are different types of servers depending on their purpose, including:

- Web servers.
- Application servers.
- Database servers.
- File servers.

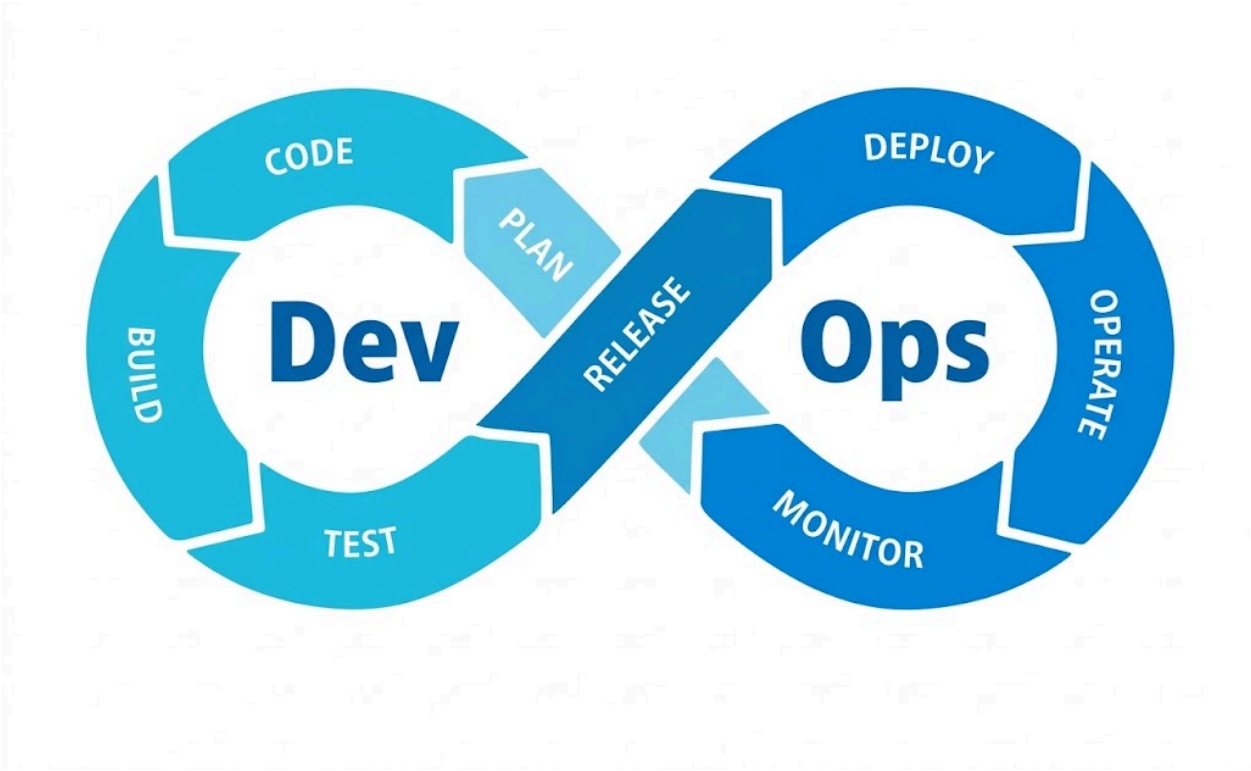
These systems are typically designed to be highly reliable, scalable, and secure, supporting multiple simultaneous connections and ensuring continuous service availability. In cloud computing, servers can be virtual, allowing greater flexibility and efficiency in resource management.

Servers typically have:

- Logic.
- Business rules.
- Validations.
- Authorizations.
- Back-end services.
- Database services.
- Etc.

DevOps

It is a methodology that integrates software development (Dev) and IT operations (Ops) practices with the goal of improving collaboration and efficiency in the creation, deployment, and management of applications and services. It promotes a culture of open and continuous communication among developers, system administrators, and other involved teams, facilitating faster development cycles and higher-quality results. DevOps practices include process automation, continuous integration and continuous delivery (CI/CD), and constant monitoring of applications in production. By breaking down the traditional barriers between development and operations, DevOps enables organizations to respond more quickly to market demands, improve software reliability, and reduce the time to market for new features and updates.

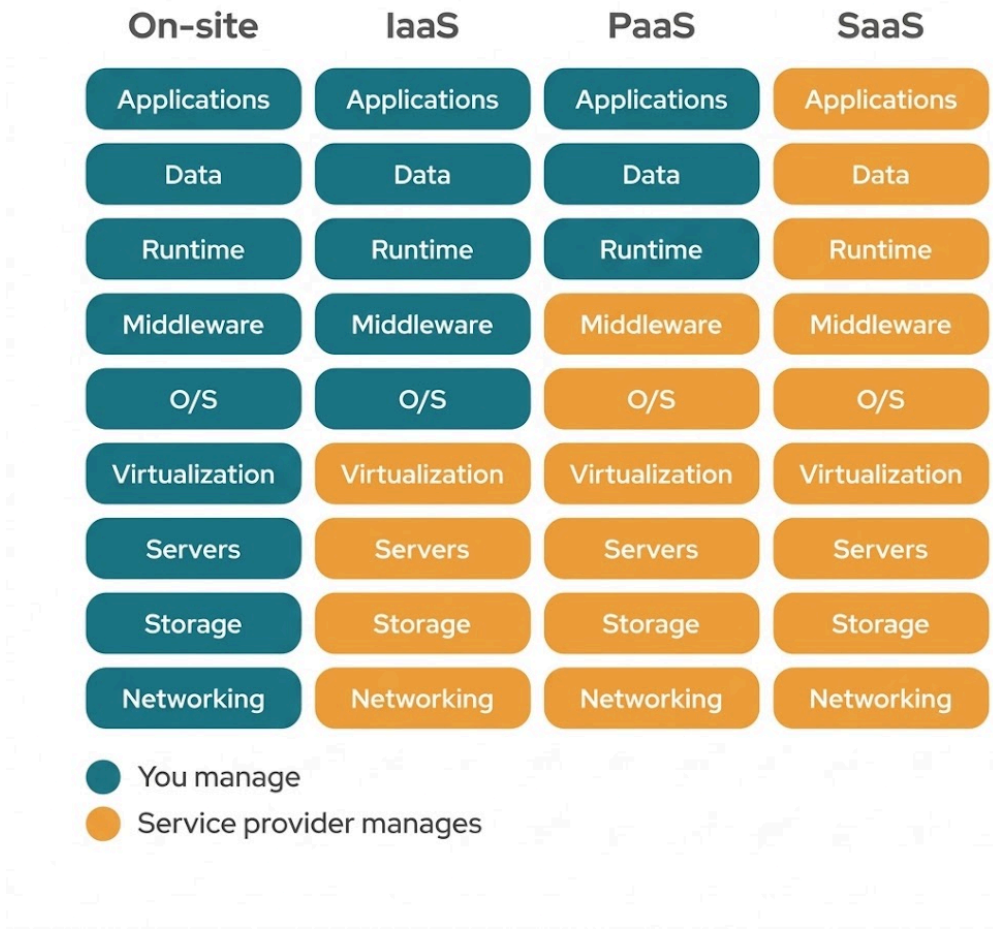


Cloud

Cloud computing refers to the delivery of computing services over the Internet, including data storage, processing, and management. Instead of using on-premises hardware and servers, organizations can access resources and applications through cloud service providers, which allows for greater flexibility, scalability, and cost savings:

- IaaS (Infrastructure as a Service).
- PaaS (Platform as a Service).
- SaaS (Software as a Service).

This technology allows companies and developers to focus on innovation and the development of their applications without worrying about the underlying infrastructure, facilitating access to powerful computing capabilities on demand and improving operational efficiency.



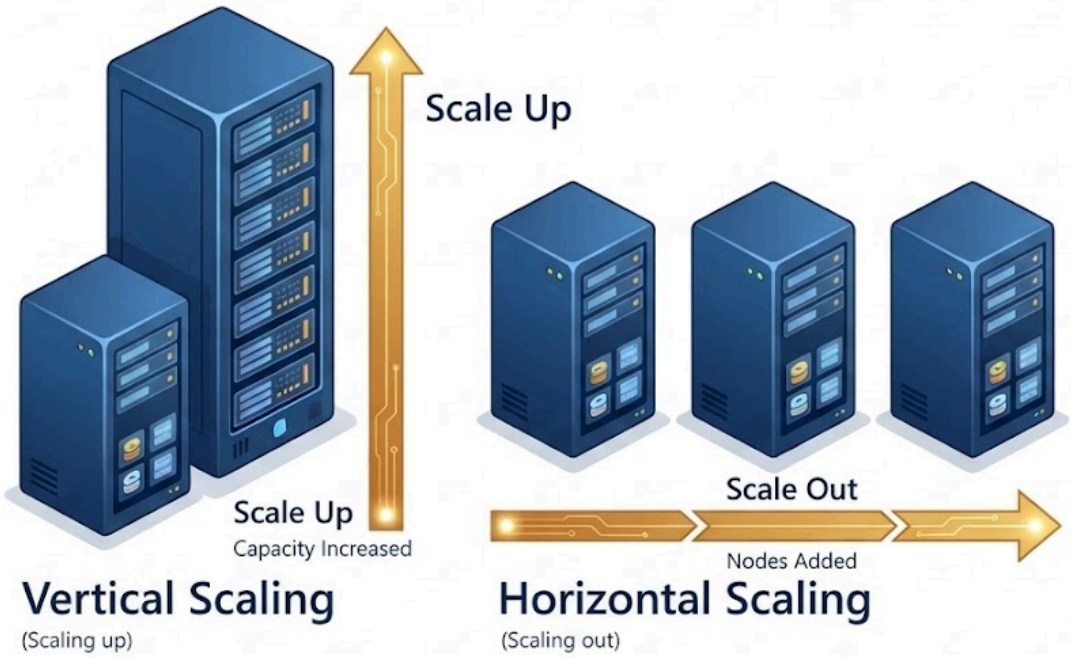
Scalability and Performance

Vertical scaling

It is a strategy for improving a system's performance by increasing the capacity of individual resources. This is achieved by adding more power to an existing server through the addition of more CPUs, RAM, or storage. Vertical scaling is a straightforward solution for increasing processing capacity and handling more intensive workloads without changing the system's architecture. However, it has physical and cost limitations, as a point is eventually reached where it is no longer possible to add more resources to a single server or where doing so becomes too expensive. Despite this, vertical scaling is simple to implement and can be effective for certain applications and environments where the complexity of horizontal scaling is not justified.

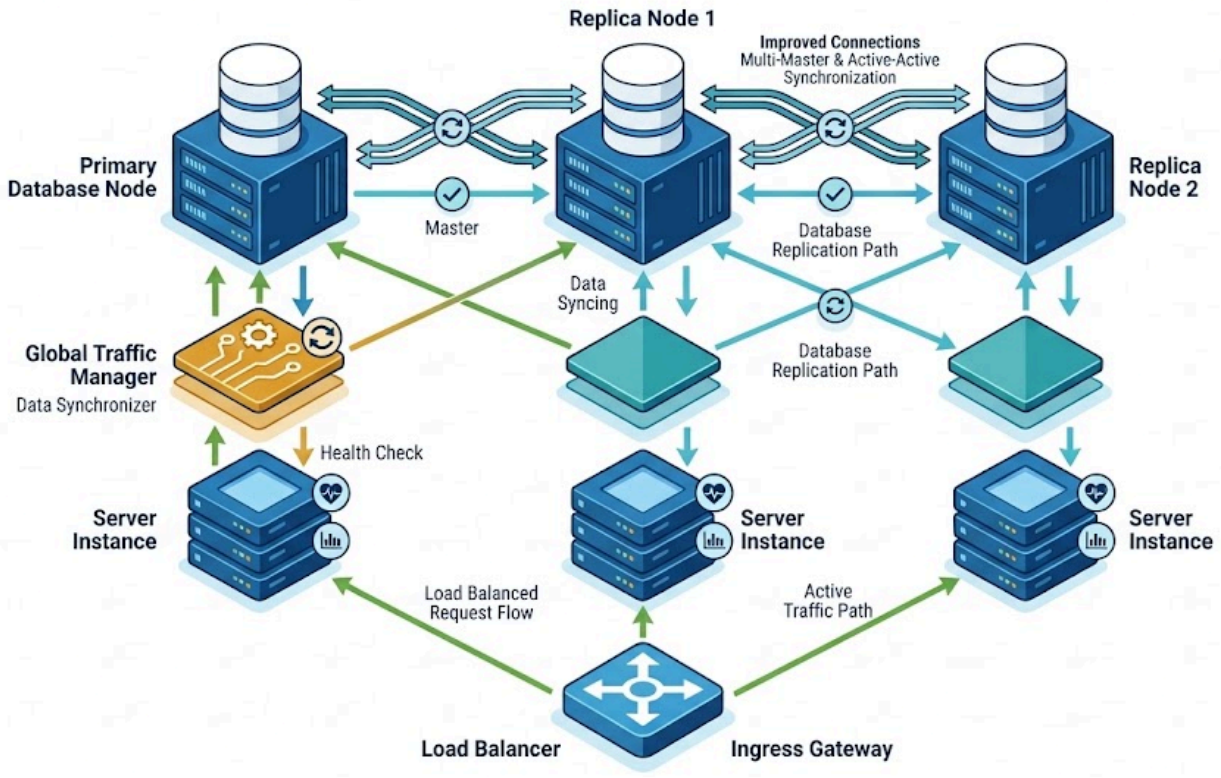
Horizontal scaling

It is a strategy for increasing a system's capacity and performance by adding more machines or nodes rather than improving the capabilities of individual resources. This approach involves distributing the workload across multiple servers, allowing for the handling of a greater volume of traffic and data processing. Horizontal scaling offers greater flexibility and redundancy, since adding new nodes can be done relatively easily and can improve fault tolerance by avoiding reliance on a single point of failure. This method is essential in microservices architectures and cloud computing environments, where resources can be dynamically adjusted according to the system's needs. Although it can be more complex to implement and manage due to the need for load balancing and synchronization between nodes, horizontal scaling is a robust and scalable solution for applications that require high availability and the ability to grow rapidly.


Scale Up
Scale Up
Capacity Increased
Vertical Scaling
(Scaling up)
Scale Out
Nodes Added
Horizontal Scaling
(Scaling out)

Replication

Replication is a process by which data is copied and kept up to date across multiple databases or servers. This mechanism is essential for improving system availability, redundancy, and performance. Replication can be synchronous, where data copies are updated simultaneously, or asynchronous, where changes are propagated at regular intervals. In high-availability environments, replication allows a replica to take over in the event of a primary server failure, ensuring service continuity. It is also used to distribute the read load across multiple nodes, thereby improving query performance. Replication is essential in distributed database systems, cloud computing, and mission-critical enterprise applications, as it ensures that data is always available and protected against loss.



Cache

It is a high-speed temporary data storage technique that enables rapid access to the most frequently requested information. By storing data in a memory that is faster and more efficiently accessible than the original data source, the cache significantly improves system performance and reduces latency. Caching systems can be implemented at various levels, including processor caches, disk caches, and distributed caches in networks and web applications. In the context of web development, caching is commonly used to store database query responses, results of frequent calculations, and static content, thereby reducing the load on servers and improving the user experience. However, it is crucial to properly manage cache invalidation and consistency to ensure that users receive up-to-date and accurate data.

Systems that implement caching:

- E-commerce.
- News sites.
- Blogs.
- DDoS attack prevention.

Task Queues

It is a process management mechanism that enables the asynchronous execution of tasks in the background. This system queues pending tasks and distributes them to dedicated workers for processing, optimizing application performance and efficiency. Task queues are essential in systems that require high-volume processing, as they allow complex or time-intensive operations to be broken down into manageable parts, preventing deadlocks and improving system responsiveness. Tools such as:

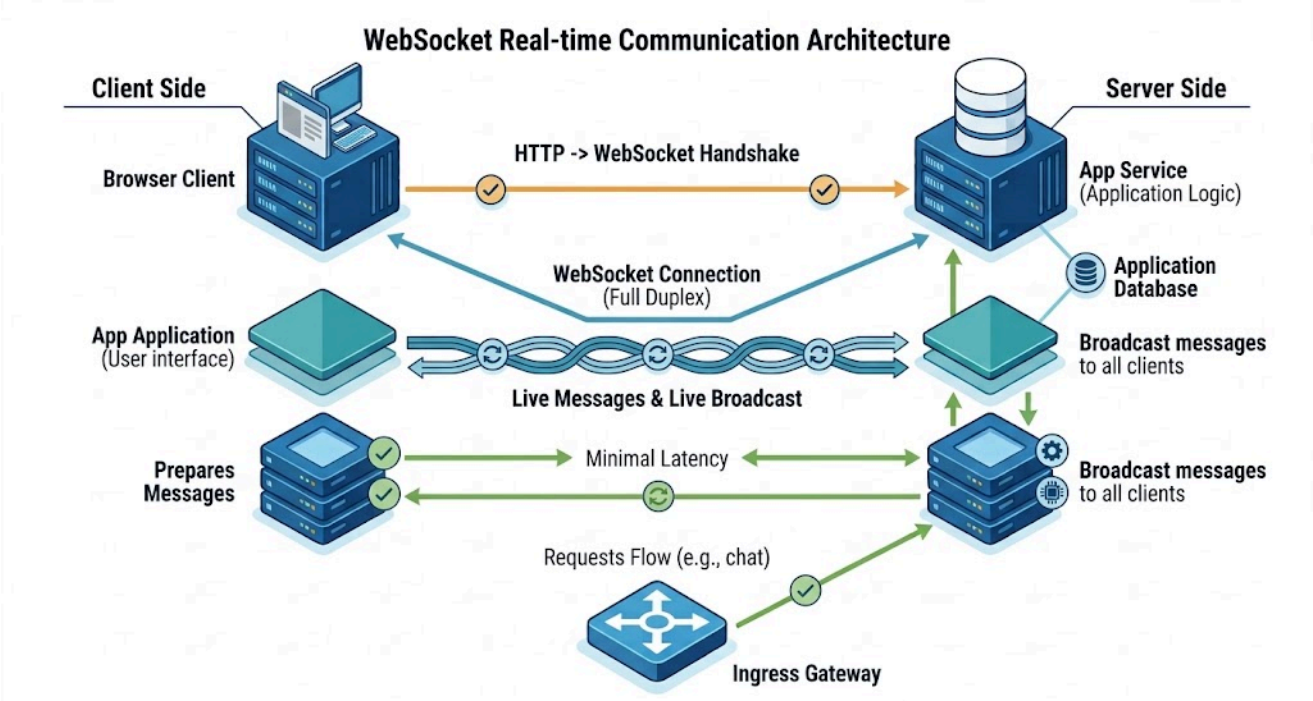
- RabbitMQ.
- Apache Kafka.
- Celery.

They are popular for implementing task queues, providing features such as persistence, redundancy, and scalability. By using task queues, developers can ensure that critical operations are performed in an orderly and reliable manner, distributing the workload and enabling automatic recovery in the event of failures.

Real-time Communication

WebSockets

WebSockets are a communication protocol that enables persistent, bidirectional communication between a client and a server over a single connection. Unlike the traditional HTTP request-response model, where the client generally initiates each communication, WebSockets allow both the client and server to send messages at any time after the connection has been established. This makes WebSockets particularly useful for applications that require real-time communication, such as chat applications, live notifications, multiplayer games, collaborative applications, and real-time dashboards.



Socket.IO

Socket.IO is a JavaScript library that provides an abstraction for building real-time, event-driven applications between clients and servers. It can use WebSockets as one of its transport mechanisms and provides additional features such as automatic reconnection, event-based communication, rooms, namespaces, and fallback mechanisms when a direct WebSocket connection is not available. Socket.IO is commonly used in Node.js applications when developers need to implement real-time features without managing all of the lower-level communication details directly.

Server implementation

```
import { WebSocketServer } from 'ws';

// Initialize WebSocket server on port 8080
const wss = new WebSocketServer({ port: 8080 });

console.log('WebSocket server running on ws://localhost:8080');

wss.on('connection', (ws) => {
    console.log('Client connected');

    // Handle incoming client messages
    ws.on('message', (data) => {
        console.log(`Received: ${data}`);

        // Send full-duplex response
        ws.send(`Server Echo: ${data}`);
    });

    // Handle client disconnect
    ws.on('close', () => console.log('Client disconnected'));
});
```

Client implementation

```
import WebSocket from 'ws';

// Initiate HTTP Upgrade Handshake
const ws = new WebSocket('ws://localhost:8080');

// Connection established
ws.on('open', () => {
    console.log('Connected to server');

    // Send message
    ws.send('Hello Server!');
});

// Receive real-time message
ws.on('message', (data) => {
    console.log(`Received from server: ${data}`);

    // Graceful closure
    ws.close();
});

// Connection closed
ws.on('close', () => console.log('Connection closed'));
```